

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, roc_auc_score, ConfusionMatrixDisplay
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

# Loading the dataset to a Pandas Dataframe
credit_card_data = pd.read_csv('/content/creditcard.csv')

# Let's see first 5 rows of the dataset
print(credit_card_data.head(5))

# Let's see last 5 rows of our dataset
print(credit_card_data.tail())

# Dataset information
print(credit_card_data.info())

# Checking number of missing values
print(credit_card_data.isnull().sum())

# Find distribution of Normal transaction or Fraud transaction
print(credit_card_data['Class'].value_counts())

# Separating the data
normal = credit_card_data[credit_card_data.Class == 0]
fraud = credit_card_data[credit_card_data.Class == 1]

print(normal.shape)
print(fraud.shape)

# Visualize the data
labels = ["Normal", "Fraud"]
count_classes = credit_card_data['Class'].value_counts()
count_classes.plot(kind="bar", rot=0)
plt.title("ProjectGurukul")

plt.ylabel("Count")
plt.xticks(range(2), labels)
plt.show()

# Statistical measures of the data
print(normal.Amount.describe())
print(fraud.Amount.describe())

# Visualize the data
sns.relplot(x='Amount', y='Time', hue='Class', data=credit_card_data)

# Compare values of both transactions
```

```

print(credit_card_data.groupby('Class').mean())

# Now we will build a sample dataset containing similar distribution of normal transaction and fraud transaction
normal_sample = normal.sample(n=492)

# Concatenate two dataframes
credit_card_new_data = pd.concat([normal_sample, fraud], axis=0)
print(credit_card_new_data['Class'].value_counts()) # Splitting data into features and targets
X = credit_card_new_data.drop('Class', axis=1)
Y = credit_card_new_data['Class']

# Splitting the data into training and testing data
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, stratify=Y, random_state=2)
print(X.shape, X_train.shape, X_test.shape) # Creating Model
model = LogisticRegression()

# Training the Logistic Regression model with training data model.fit(X_train, Y_train)
# === Models ===
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(random_state=2),
    "Random Forest": RandomForestClassifier(random_state=2),
    "Support Vector Machine": SVC(probability=True, random_state=2)
}

# Train and evaluate each model
for name, model in models.items():
    print(f"\n=== {name} ===")
    model.fit(X_train, Y_train)

    # Predict
    Y_train_pred = model.predict(X_train)
    Y_test_pred = model.predict(X_test)
    Y_test_proba = model.predict_proba(X_test)[: , 1]

    # Metrics
    train_acc = accuracy_score(Y_train, Y_train_pred)
    test_acc = accuracy_score(Y_test, Y_test_pred)
    roc_auc = roc_auc_score(Y_test, Y_test_proba)

    print("Training Accuracy:", train_acc)
    print("Testing Accuracy :", test_acc)
    print("ROC-AUC Score  :", roc_auc)

    # Confusion Matrix
    cm = confusion_matrix(Y_test, Y_test_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Normal', 'Fraud'])
    disp.plot()
    plt.title(f"Confusion Matrix - {name}")
    plt.show()

    # Model Evaluation
    X_train_pred = model.predict(X_train)
    training_data_accuracy = accuracy_score(X_train_pred, Y_train)
    print('Accuracy of Training data:', training_data_accuracy)

    # Accuracy on test data

```

```
X_test_pred = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_pred, Y_test)
print('Accuracy of Testing data:', test_data_accuracy)

# Confusion matrix and classification report
print(confusion_matrix(Y_test, X_test_pred))
print(classification_report(Y_test, X_test_pred))

# Anomaly Detection using Isolation Forest
iso_forest = IsolationForest(contamination=0.01, random_state=42)
X_iso = credit_card_data.drop('Class', axis=1) # Use features only
y_iso = iso_forest.fit_predict(X_iso)

# Convert predictions to binary (1 for normal, -1 for anomaly)
y_iso = np.where(y_iso == -1, 1, 0)

# Add predictions to the original dataframe
credit_card_data['Anomaly'] = y_iso

# Visualize the anomalies
plt.figure(figsize=(10, 6))
sns.scatterplot(data=credit_card_data, x='Amount', y='Time', hue='Anomaly', palette={0: 'blue', 1: 'red'}, alpha=0.5)
plt.title('Anomaly Detection in Credit Card Transactions')
plt.xlabel('Amount')
plt.ylabel('Time')
plt.legend(title='Anomaly', loc='upper right', labels=['Normal', 'Anomaly'])
plt.show()

# Plot combined ROC curve
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title("ROC Curve Comparison")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.grid()
plt.show()
```

```

Time      V1      V2      V3      V4      V5      V6      V7  \
0  0.0 -1.359807 -0.072781 2.536347 1.378155 -0.338321 0.462388 0.239599
1  0.0  1.191857  0.266151 0.166480 0.448154 0.060018 -0.082361 -0.078803
2  1.0 -1.358354 -1.340163 1.773209 0.379780 -0.503198 1.800499 0.791461
3  1.0 -0.966272 -0.185226 1.792993 -0.863291 -0.010309 1.247203 0.237609
4  2.0 -1.158233  0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941

```

```

V8      V9      ...      V21      V22      V23      V24      V25  \
0  0.098698 0.363787 ... -0.018307 0.277838 -0.110474 0.066928 0.128539
1  0.085102 -0.255425 ... -0.225775 -0.638672 0.101288 -0.339846 0.167170
2  0.247676 -1.514654 ... 0.247998 0.771679 0.909412 -0.689281 -0.327642
3  0.377436 -1.387024 ... -0.108300 0.005274 -0.190321 -1.175575 0.647376
4 -0.270533 0.817739 ... -0.009431 0.798278 -0.137458 0.141267 -0.206010

```

```

V26      V27      V28  Amount  Class
0 -0.189115 0.133558 -0.021053 149.62 0
1  0.125895 -0.008983 0.014724 2.69 0
2 -0.139097 -0.055353 -0.059752 378.66 0
3 -0.221929 0.062723 0.061458 123.50 0
4  0.502292 0.219422 0.215153 69.99 0

```

[5 rows x 31 columns]

```

Time      V1      V2      V3      V4      V5  \
284802 172786.0 -11.881118 10.071785 -9.834783 -2.066656 -5.364473
284803 172787.0 -0.732789 -0.055080 2.035030 -0.738589 0.868229
284804 172788.0 1.919565 -0.301254 -3.249640 -0.557828 2.630515
284805 172788.0 -0.240440 0.530483 0.702510 0.689799 -0.377961
284806 172792.0 -0.533413 -0.189733 0.703337 -0.506271 -0.012546

```

```

V6      V7      V8      V9      ...      V21      V22  \
284802 -2.606837 -4.918215 7.305334 1.914428 ... 0.213454 0.111864
284803 1.058415 0.024330 0.294869 0.584800 ... 0.214205 0.924384
284804 3.031260 -0.296827 0.708417 0.432454 ... 0.232045 0.578229
284805 0.623708 -0.686180 0.679145 0.392087 ... 0.265245 0.800049
284806 -0.649617 1.577006 -0.414650 0.486180 ... 0.261057 0.643078

```

```

V23      V24      V25      V26      V27      V28  Amount  \
284802 1.014480 -0.509348 1.436807 0.250034 0.943651 0.823731 0.77
284803 0.012463 -1.016226 -0.606624 -0.395255 0.068472 -0.053527 24.79
284804 -0.037501 0.640134 0.265745 -0.087371 0.004455 -0.026561 67.88
284805 -0.163298 0.123205 -0.569159 0.546668 0.108821 0.104533 10.00
284806 0.376777 0.008797 -0.473649 -0.818267 -0.002415 0.013649 217.00

```

```

Class
284802 0
284803 0
284804 0
284805 0
284806 0

```

[5 rows x 31 columns]

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 284807 entries, 0 to 284806
```

```
Data columns (total 31 columns):
```

```

#   Column  Non-Null Count  Dtype
---  -
0   Time    284807 non-null    float64
1   V1       284807 non-null    float64
2   V2       284807 non-null    float64
3   V3       284807 non-null    float64
4   V4       284807 non-null    float64
5   V5       284807 non-null    float64

```